

Kuggle 2차 프로젝트 회귀분석

회귀분석을 이용한 주택 가격 예측

김재환 김유빈 손재강 이재현 남주혜 이도윤

TABLE OF CONTENTS

목차

01 주제 및 분석 개요

02 데이터 전처리

03 선형 회귀 분석

04 머신러닝

05 분석 마무리

06 결론

주제 및 분석 개요

주제

주택 가격은 면적이나 연식 같은 단순한 수치를 넘어 지리적 위치, 건축 등급 등 수많은 요인이 비선형적으로 얹혀 결정됩니다. 이를 객관적으로 분석하는 과정을 통해 가장 우수한 예측 성능을 지닌 모델을 발굴하는 동시에, 머신러닝의 변수 중요도를 파악하여 주택의 물리적 스펙과 세부 입지 조건이 자산 가치에 미치는 영향을 정량적으로 규명하고자 합니다.

Columns — House Sales in King County_USA

- | | |
|---------------|-----------------|
| - id | - grade |
| - date | - sqft_above |
| - price | - sqft_basement |
| - bedrooms | - yr_built |
| - bathrooms | - yr_renovated |
| - sqft_living | - zipcode |
| - sqft_lot | - lat |
| - floors | - long |
| - waterfront | - sqft_living15 |
| - view | - sqft_lot15 |
| - condition | |

데이터 전처리

데이터 전처리

```

1  import pandas as pd
2
3  df = pd.read_csv("House Sales in King County_USA.csv")
4
5  # 불필요한 컬럼 id 제거
6  df = df.drop(['id'], axis=1)
7
8  # 날짜(date) 데이터 처리 및 파생변수 생성 - '20141013T000000' 형태의 문자를 날짜형으로 변환
9  df['date'] = pd.to_datetime(df['date'])
10
11 # 거래 연도와 월을 추출해서 새로운 컬럼으로 만들 - 부동산은 거래 시기 중요
12 df['year_sold'] = df['date'].dt.year
13 df['month_sold'] = df['date'].dt.month
14 df = df.drop(['date'], axis=1)

```

1 불러오기

- 데이터셋을 불러오고 불필요한 컬럼을 제거함
- 부동산은 거래 시기가 중요하기에 거래 연월을 추출하여 파생변수를 만듦

데이터 전처리

데이터 전처리

```

1  # 중요한 파생 변수(Feature Engineering) 추가 - 집의 실제 나이 계산: (거래된
2  df['house_age'] = df['year_sold'] - df['yr_built']
3
4  # 리모델링 여부: 리모델링 연도가 0보다 크면 1(했음), 0이면 0(안했음)으로 이
5  df['is_renovated'] = df['yr_renovated'].apply(lambda x: 1 if x > 0 else 0)
6
7  # 5. 이상치(Outlier) 제거
8  # 데이터 확인 결과 침실(bedrooms)이 33개인 집이 1채 있음. 오타일 확률이 매우
9  df = df[df['bedrooms'] < 33]
10
11 df.to_csv("House_Sales_Cleaned.csv", index=False)

```

2 파생변수 생성

- 집의 실제 나이를 계산한 파생변수를 만듦
- 'renovated' 여부를 0과 1로 이원화한 후, 이상치를 제거함

데이터 전처리

데이터 전처리

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.model_selection import train_test_split
5  df = pd.read_csv("House_Sales_Cleaned.csv")
6
7  # 비대칭 분포 데이터 로그 변환
8  # 타겟 변수인 price와 변동폭이 큰 면적 데이터에 np.log1p 적용
9  df['price_log'] = np.log1p(df['price'])
10 df['sqft_living_log'] = np.log1p(df['sqft_living'])
11 df['sqft_lot_log'] = np.log1p(df['sqft_lot'])
12
13 df = df.drop(['sqft_living', 'sqft_lot'], axis=1)

```

3 로그 변환

- 전처리에 필요한 모듈들을 불러오고, 타겟변수와 변동폭이 큰 데이터에 로그를 씌움
- 로그를 씌운 변수를 생성했기 때문에 기존의 면적 데이터는 제거함

데이터 전처리

데이터 전처리

```

16 # 범주형 변수(zipcode) 원-핫 인코딩
17 df['zipcode'] = df['zipcode'].astype(str)
18 df = pd.get_dummies(df, columns=['zipcode'], drop_first=True)
19
20
21 # 데이터 스케일링 (StandardScaler)
22 # 원-핫 인코딩된 변수와 타겟 변수(price_log)를 제외한 연속형 수치 변수만 스케일링
23 features_to_scale = [
24     'bedrooms', 'bathrooms', 'sqft_above', 'sqft_basement',
25     'sqft_living15', 'sqft_lot15', 'house_age', 'condition', 'grade',
26     'sqft_living_log', 'sqft_lot_log'
27 ]
28
29 scaler = StandardScaler()
30 df[features_to_scale] = scaler.fit_transform(df[features_to_scale])

```

4 인코딩 및 스케일링

- 'zipcode'의 경우 지역을 의미하는 범주형 변수이므로 문자열 데이터로 변경하고 원-핫 인코딩을 진행함
- 수치형 변수들을 스케일링을 통해 평균을 0, 표준편차를 1로 맞춰 단위를 통일함

데이터 전처리

데이터 전처리

```

1  # Train / Test 데이터 분리
2  # 예측타겟은 정규화된 price_log 사용
3  X = df.drop(['price', 'price_log'], axis=1)
4  y = df['price_log']
5
6  # 학습용 데이터 80%, 테스트용 데이터 20%로 분할
7  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
8
9  print("전처리 완료, 데이터 분할")
10 print(f"X_train의 형태: {X_train.shape}, X_test의 형태: {X_test.shape}")

```

전처리 완료, 데이터 분할

X_train의 형태: (17289, 90), X_test의 형태: (4323, 90)

5 Train/Test data

- 가격을 제외한 독립변수들로 X를 만들고, 타겟변수를 로그 변환한 가격으로 지정함
- 훈련용 데이터를 80%, 테스트 데이터를 20%로 분할함

선형 회귀 분석

OLS (다중 선형 회귀 분석)

```

1  # OLS 모델은 절편(intercept)을 자동으로 추가하지 않으므로, 수동으로 추가합니다.
2  X_train_const = sm.add_constant(X_train)
3  X_test_const = sm.add_constant(X_test)

1  import numpy as np
2  # 3. OLS 모델 학습 및 결과 요약
3  # -----
4
5  # Convert boolean columns to integer (0 or 1) type
6  X_train_const = X_train_const.astype(float)
7  X_test_const = X_test_const.astype(float)
8
9  # OLS 모델 생성 및 학습
10 model = sm.OLS(y_train, X_train_const)
11 results = model.fit()

```

1 OLS 모델 학습

- OLS 모델은 절편을 자동으로 추가하지 않으므로 수동으로 추가함
- 모든 데이터를 실수형으로 변환하고 OLS 회귀 모델을 생성하여 실제 데이터에 맞춰 학습시킴

선형 회귀 분석

OLS (다중 선형 회귀 분석)

```
1 # 회귀분석 결과 요약 출력
2 print("\n" + "="*80)
3 print(results.summary())
4 print("="*80 + "\n")
```

숨겨진 출력 표시

```
=====
                        OLS Regression Results
=====
Dep. Variable:          price_log    R-squared:                0.888
Model:                  OLS          Adj. R-squared:            0.887
Method:                 Least Squares    F-statistic:              1526.
Date:                   Thu, 21 May 2026    Prob (F-statistic):        0.00
Time:                   10:01:07          Log-Likelihood:           5511.5
No. Observations:       17289             AIC:                     -1.084e+04
Df Residuals:           17199             BIC:                     -1.014e+04
Df Model:                89
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	-67.6735	3.800	-17.811	0.000	-75.121	-60.226
bedrooms	-0.0132	0.002	-7.072	0.000	-0.017	-0.010
bathrooms	0.0303	0.003	12.093	0.000	0.025	0.035
floors	-0.0174	0.004	-4.283	0.000	-0.025	-0.009
waterfront	0.4737	0.018	26.872	0.000	0.439	0.508
view	0.0599	0.002	27.629	0.000	0.056	0.064
condition	0.0381	0.002	24.611	0.000	0.035	0.041

2 회귀분석 결과 요약

- 학습이 완료된 모델의 종합적인 통계 결과표를 출력함
- 모델의 설명력은 88.8%로 주택 가격 변동의 대부분을 훌륭하게 설명하고 있으며 모형의 통계적 유의성도 확보됨

선형 회귀 분석

OLS (다중 선형 회귀 분석)

```
# 모델의 예측값(fitted values) 및 잔차(residuals) 계산 (학습 데이터 기준)
fitted_values = results.fittedvalues
residuals = results.resid

# 테스트 데이터에 대한 예측
predictions_test = results.predict(X_test_const)

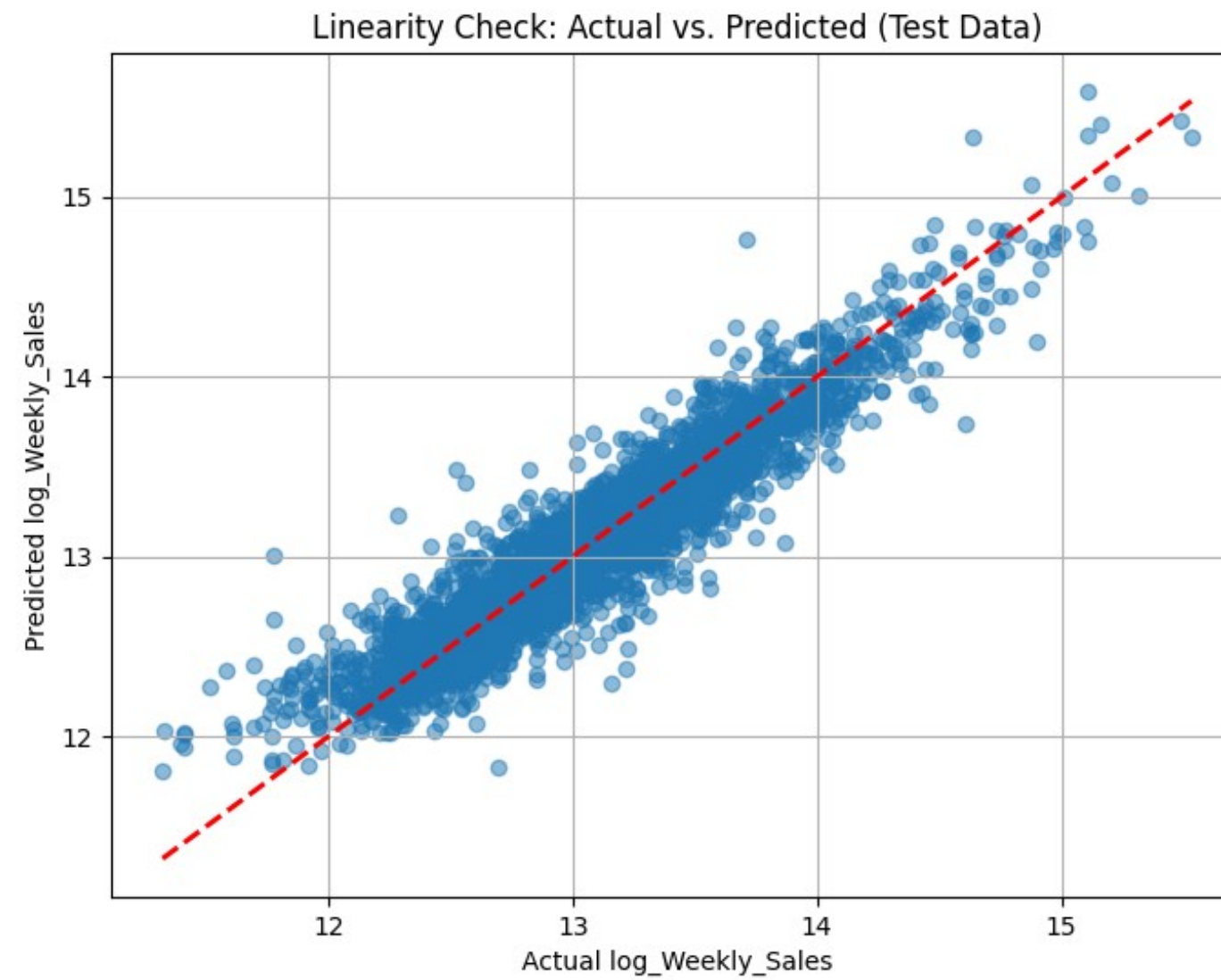
# 1. 선형성 (Linearity) 확인
# - 예측값(test)과 실제값(test)을 비교하는 Scatter plot
print("\n--- 1. 선형성 검증 (Actual vs. Predicted) ---")
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predictions_test, alpha=0.5)
# 추세선 (y=x)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Linearity Check: Actual vs. Predicted (Test Data)')
plt.xlabel('Actual log_Weekly_Sales')
plt.ylabel('Predicted log_Weekly_Sales')
plt.grid(True)
plt.savefig("linearity_check.png")
plt.close() # Close plot to prevent showing in output
print(" 'linearity_check.png' (선형성 검증) 플롯이 생성되었습니다.")
```

3 예측 및 선형성 검증

- 모델의 예측값과 잔차를 추출하고 테스트 데이터를 넣어 최종 가격 예측을 함
- 실제값과 예측값을 각각 점으로 찍어 산점도를 그리고 $y=x$ 형태의 빨간색 점선을 나타냄

선형 회귀 분석

OLS (다중 선형 회귀 분석)



선형 회귀 분석

OLS (다중 선형 회귀 분석)

```

1  # 2. 잔차의 정규성 (Normality of Residuals) 확인
2  # - Q-Q Plot
3  print("\n--- 2. 잔차의 정규성 검증 (Q-Q Plot) ---")
4  plt.figure(figsize=(8, 6))
5  stats.probplot(residuals, dist="norm", plot=plt)
6  plt.title('Normality Check: Q-Q Plot of Residuals')
7  plt.grid(True)
8  plt.savefig("qq_plot.png")
9  plt.close() # Close plot
10 print(" 'qq_plot.png' (Q-Q 플롯)이 생성되었습니다.")

```

```

-- 2. 잔차의 정규성 검증 (Q-Q Plot) ---
'qq_plot.png' (Q-Q 플롯)이 생성되었습니다.

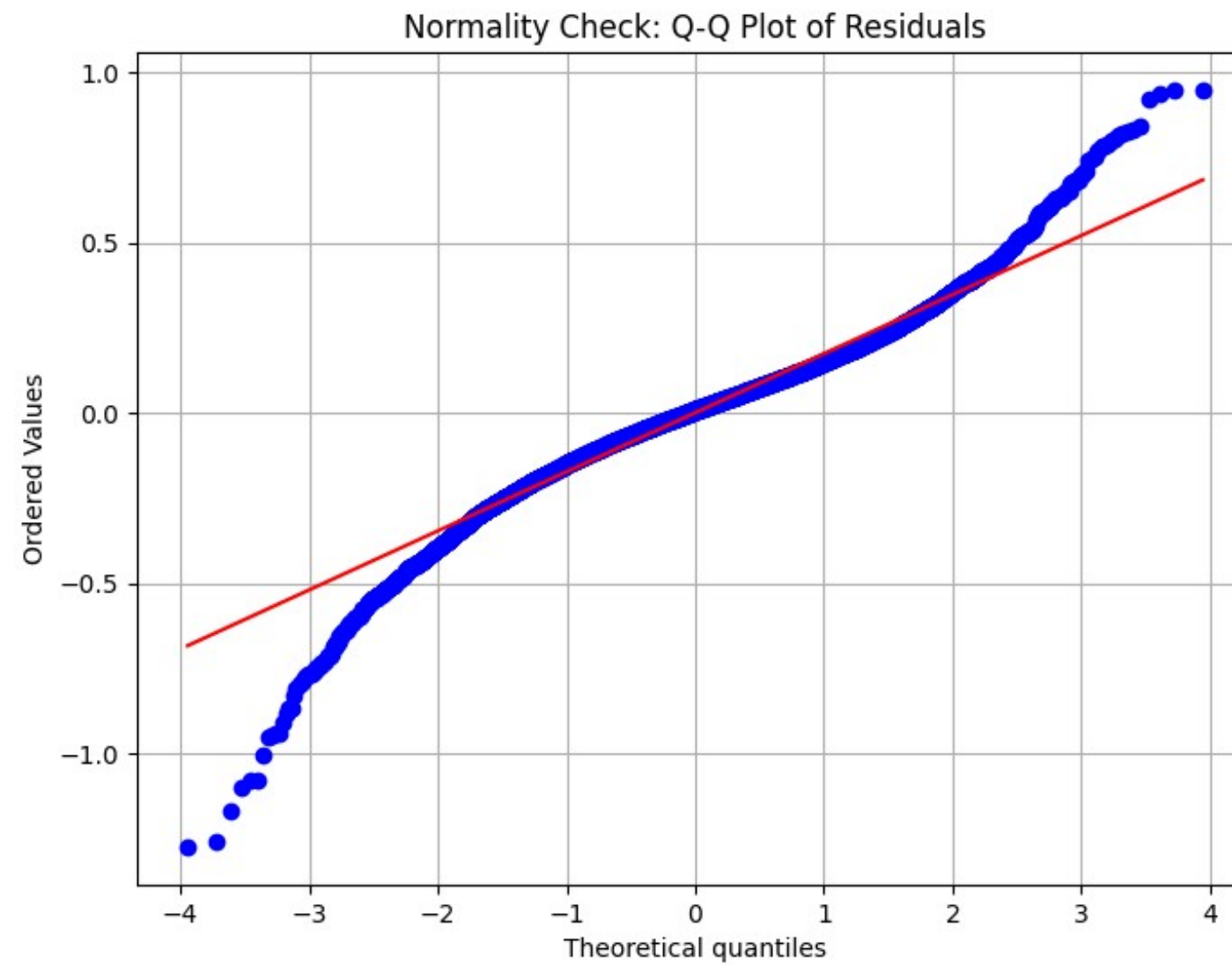
```

4 잔차의 정규성 검증

- 잔차들이 정규분포를 따르는지 확인하기 위해 Q-Q 플롯을 생성함
- 데이터 점들이 빨간색 45도 대각선에 가깝게 달라붙어 있을수록 잔차가 정규분포를 잘 따른다고 판단함

선형 회귀 분석

OLS (다중 선형 회귀 분석)



선형 회귀 분석

OLS (다중 선형 회귀 분석)

```

1  # 3. 잔차의 등분산성 (Homoscedasticity) 확인
2  # - 예측값(fitted values)과 잔차(residuals)의 Scatter plot
3  print("\n--- 3. 잔차의 등분산성 검증 (Residual Plot) ---")
4  plt.figure(figsize=(8, 6))
5  plt.scatter(fitted_values, residuals, alpha=0.5)
6  # 0 기준선
7  plt.axhline(y=0, color='r', linestyle='--', lw=2)
8  plt.title('Homoscedasticity Check: Fitted Values vs. Residuals')
9  plt.xlabel('Fitted Values')
10 plt.ylabel('Residuals')
11 plt.grid(True)
12 plt.savefig("residual_plot.png")
13 plt.close() # Close plot
14 print(" 'residual_plot.png' (잔차 플롯)이 생성되었습니다.")

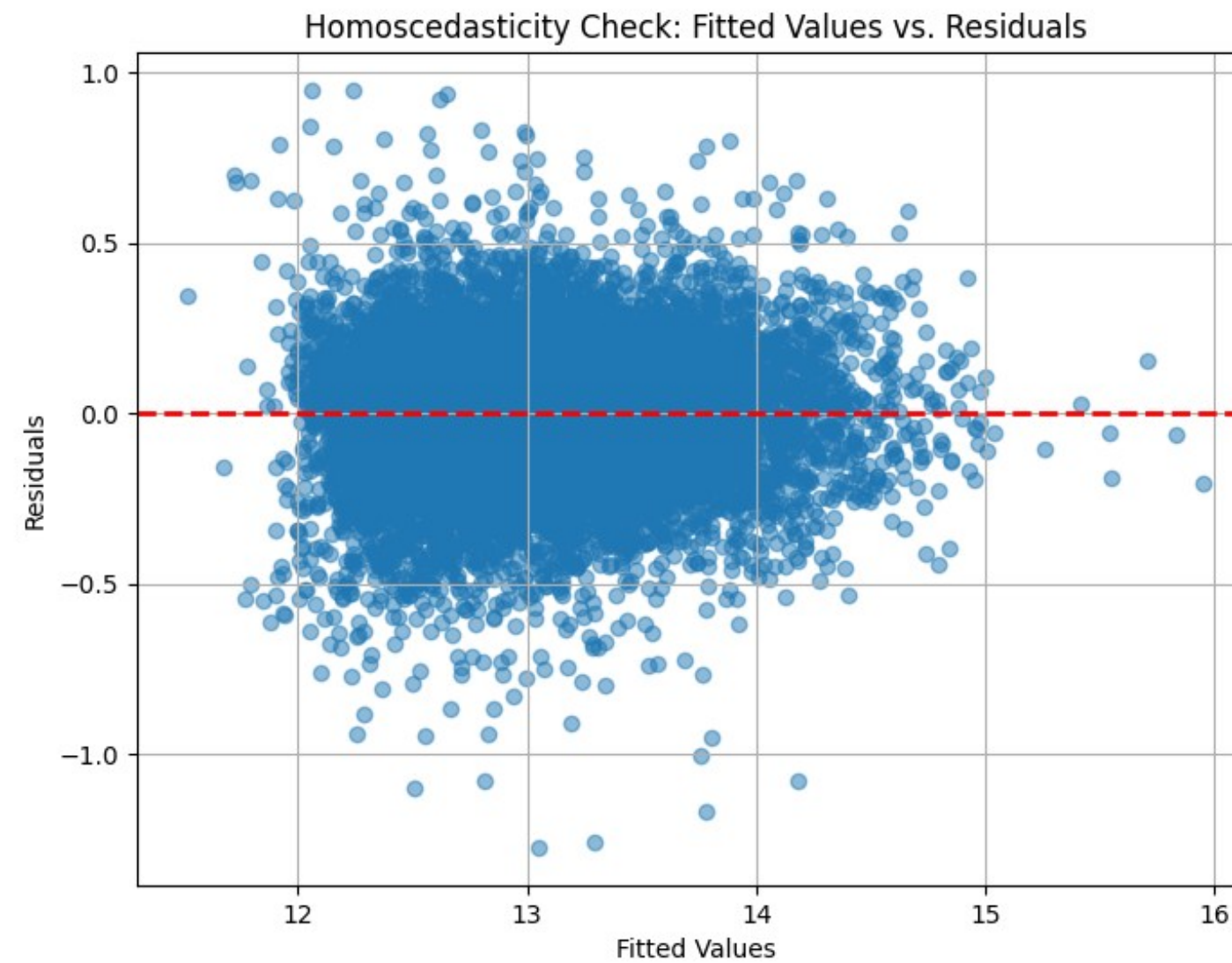
```

5 잔차 등분산성 검증

- X축에는 모델의 예측값,
Y축에는 잔차로 산점도를
생성하고 Y축의 0 위치에
빨간색 수평 점선을 그음
- 잔차들이 0을 기준으로 위아래
무작위로 고르게 흩뿌려져
있어야 등분산성을 만족함

선형 회귀 분석

OLS (다중 선형 회귀 분석)



선형 회귀 분석

OLS (다중 선형 회귀 분석)

```
# 4. 다중공선성 (Multicollinearity) 확인
# - VIF (Variance Inflation Factor) 계산
# - VIF는 독립변수들(X_train)만으로 계산합니다. (상수항 제외)
print("\n--- 4. 다중공선성 검증 (VIF) ---")
vif_data = pd.DataFrame()
vif_data["feature"] = X_train.columns

# Convert X_train to float type to ensure numerical processing for VIF calculation
X_train_numeric = X_train.astype(float)
vif_data["VIF"] = [variance_inflation_factor(X_train_numeric.values, i) for i in range(len(X_train_numeric.columns))]

print(vif_data)
print("\nVIF 해석:")
print("- VIF가 10 이상이면 다중공선성이 높다고 판단합니다 (일부 기준에서는 5).")
print("- VIF가 1에 가까울수록 다중공선성 문제가 적습니다.")
print("\n--- 모든 분석 완료 ---")
```

6 다중공선성 검증

- VIF 수치를 저장할 빈 표를 만들고 'feature' 컬럼에 독립 변수들의 이름을 넣음
- 반복문을 사용해 모든 독립 변수를 하나씩 순회하며 VIF 값을 계산하여 'VIF' 컬럼에 채움

선형 회귀 분석

OLS (다중 선형 회귀 분석)

--- 4. 다중공선성 검증 (VIF) ---

	feature	VIF
0	bedrooms	1.925417
1	bathrooms	3.478051
2	floors	2.713676
3	waterfront	1.259262
4	view	1.513224
...	...	...
85	zipcode_98177	9.110616
86	zipcode_98178	3.235344
87	zipcode_98188	1.768495
88	zipcode_98198	2.066126
89	zipcode_98199	7.558686

[90 rows x 2 columns]

VIF 해석:

- VIF가 10 이상이면 다중공선성이 높다고 판단합니다 (일부 기준에서는 5).
- VIF가 1에 가까울수록 다중공선성 문제가 적습니다.

--- 모든 분석 완료 ---

선형 회귀 분석

Ridge — 모델의 가중치가 커지는 것을 방지하기 위한 L2 규제 사용

```

3 # RidgeCV는 교차 검증(Cross-Validation)을 통해 최적의 alpha 값을 자동으로 찾아줍니다.
4 # 테스트할 alpha 값들의 범위 설정
5 alphas_ridge = np.logspace(-3, 3, 100) # 10^-3 에서 10^3 까지 100개
6
7 ridge_cv = RidgeCV(alphas=alphas_ridge, cv=5)
8 ridge_cv.fit(X_train, y_train)
9
10 # 최적의 alpha
11 print(f"\n--- Ridge Regression (RidgeCV) ---")
12 print(f"Best Alpha: {ridge_cv.alpha_:.4f}")
13
14 # 테스트 R-squared
15 ridge_r2_test = ridge_cv.score(X_test, y_test)
16 print(f"Ridge Test R-squared: {ridge_r2_test:.4f}")
17
18 # Ridge 계수
19 ridge_coefs = ridge_cv.coef_

```

```

--- Ridge Regression (RidgeCV) ---
Best Alpha: 0.0011
Ridge Test R-squared: 0.8881

```

1 Ridge 회귀분석

- 교차검증을 5번 진행하는 릿지 모델을 생성하고 설정된 모델로 학습 데이터를 학습시킴
- 학습된 릿지 모델에 테스트 데이터를 넣어 예측해 보고 실제 정답과 비교하여 모델의 설명력을 계산함

선형 회귀 분석

Lasso — L1 규제로 덜 중요한 변수의 가중치를 0으로 만들어 변수 자동 선택

```

3  # LassoCV도 교차 검증으로 최적의 alpha를 찾습니다.
4  alphas_lasso = np.logspace(-6, 0, 100) # 10^-6 에서 10^0 까지 100개
5
6  lasso_cv = LassoCV(alphas=alphas_lasso, cv=5, max_iter=10000, random_state=42)
7  lasso_cv.fit(X_train, y_train)
8
9  # 최적의 alpha
10 print(f"\n--- Lasso Regression (LassoCV) ---")
11 print(f"Best Alpha: {lasso_cv.alpha_:.4f}")
12
13 # 테스트 R-squared
14 lasso_r2_test = lasso_cv.score(X_test, y_test)
15 print(f"Lasso Test R-squared: {lasso_r2_test:.4f}")
16
17 # Lasso 계수
18 lasso_coefs = lasso_cv.coef_

```

2 Lasso 회귀분석

- 라쏘 회귀모델을 생성하고
충분한 계산을 위해 반복을
1만 번으로 넉넉하게 설정함
- 교차 검증을 통해 최적의
alpha를 찾고 학습 데이터를
학습시킴

선형 회귀 분석

Ridge + Lasso

```

1  # 7. 계수 비교
2  # -----
3  print("\n" + "="*80)
4  print("                Coefficient Comparison (Scaled Data)")
5  print("="*80)
6
7  # Define 'features' correctly using the index of ols_coefs
8  features = ols_coefs.index.tolist()
9
10 coef_df = pd.DataFrame({
11     'feature': features,
12     'OLS_Scaled_Coef': ols_coefs.values,
13     'Ridge_Coef': ridge_coefs,
14     'Lasso_Coef': lasso_coefs
15 })
16
17 print(coef_df)
18 print("="*80)

```

3 계수 비교

- 기본 OLS, 릿지, 라쏘 모델이 독립변수들에 대해 각각 어떤 가중치를 부여했는지 비교하기 위해 표를 만듦
- 앞서 만든 변수 이름 리스트를 넣고 OLS 모델, 릿지 모델, 라쏘 모델의 회귀계수 값 입력

선형 회귀 분석

Ridge + Lasso

```
=====
                        Coefficient Comparison (Scaled Data)
=====
```

	feature	OLS_Scaled_Coef	Ridge_Coef	Lasso_Coef
0	bedrooms	-0.013241	-0.013232	-0.013036
1	bathrooms	0.030334	0.030355	0.030802
2	floors	-0.017423	-0.017443	-0.017843
3	waterfront	0.473695	0.473532	0.470289
4	view	0.059896	0.059888	0.059726
..	...	...	...	...
85	zipcode_98177	0.331938	0.331188	0.308735
86	zipcode_98178	0.084273	0.083933	0.072654
87	zipcode_98188	0.024898	0.024579	0.014651
88	zipcode_98198	0.001366	0.001212	-0.003997
89	zipcode_98199	0.657272	0.656708	0.639732

```

[90 rows x 4 columns]
=====
```

선형 회귀 분석

Ridge + Lasso

```

1  # 8. 가정 검증을 위한 값 계산
2  # -----
3
4  # ----- Ridge 모델 값 -----
5  # 훈련 데이터에 대한 예측값 (등분산성, 정규성 검증용)
6  fitted_values_train_ridge = ridge_cv.predict(X_train)
7  # 훈련 데이터의 잔차 (등분산성, 정규성 검증용)
8  residuals_train_ridge = y_train - fitted_values_train_ridge
9  # 테스트 데이터에 대한 예측값 (선형성 검증용)
10 predictions_test_ridge = ridge_cv.predict(X_test)
11
12 # ----- Lasso 모델 값 -----
13 # 훈련 데이터에 대한 예측값
14 fitted_values_train_lasso = lasso_cv.predict(X_train)
15 # 훈련 데이터의 잔차
16 residuals_train_lasso = y_train - fitted_values_train_lasso
17 # 테스트 데이터에 대한 예측값
18 predictions_test_lasso = lasso_cv.predict(X_test)
19
20 print("\n--- 검증용 값 계산 완료 ---")

```

4 검증용 값 계산

- 학습이 끝난 릿지와 라쏘 모델들도 기본 OLS와 같이 잔차 분석을 진행하기 위해 필요한 예측값과 잔차 값들을 계산함

선형 회귀 분석

Ridge + Lasso

```

2  # -----
3  import matplotlib.pyplot as plt
4  import scipy.stats as stats
5  # ---- 1. 선형성 (Actual vs. Predicted) ----
6  plt.figure(figsize=(12, 6))
7  plt.subplot(1, 2, 1)
8  plt.scatter(y_test, predictions_test_lasso, alpha=0.5)
9  plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
10 plt.title('Lasso: Linearity Check (Test Data)')
11 plt.xlabel('Actual')
12 plt.ylabel('Predicted')
13 plt.grid(True)
14
15 plt.subplot(1, 2, 2)
16 plt.scatter(y_test, predictions_test_lasso, alpha=0.5)
17 plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
18 plt.title('Lasso: Linearity Check (Test Data)')
19 plt.xlabel('Actual')
20 plt.ylabel('Predicted')
21 plt.grid(True)
22 plt.tight_layout()
23 plt.savefig("ridge_lasso_linearity.png")
24 print(" 'ridge_lasso_linearity.png' (선형성 검증) 플롯이 생성되었습니다.")
25 # plt.show()

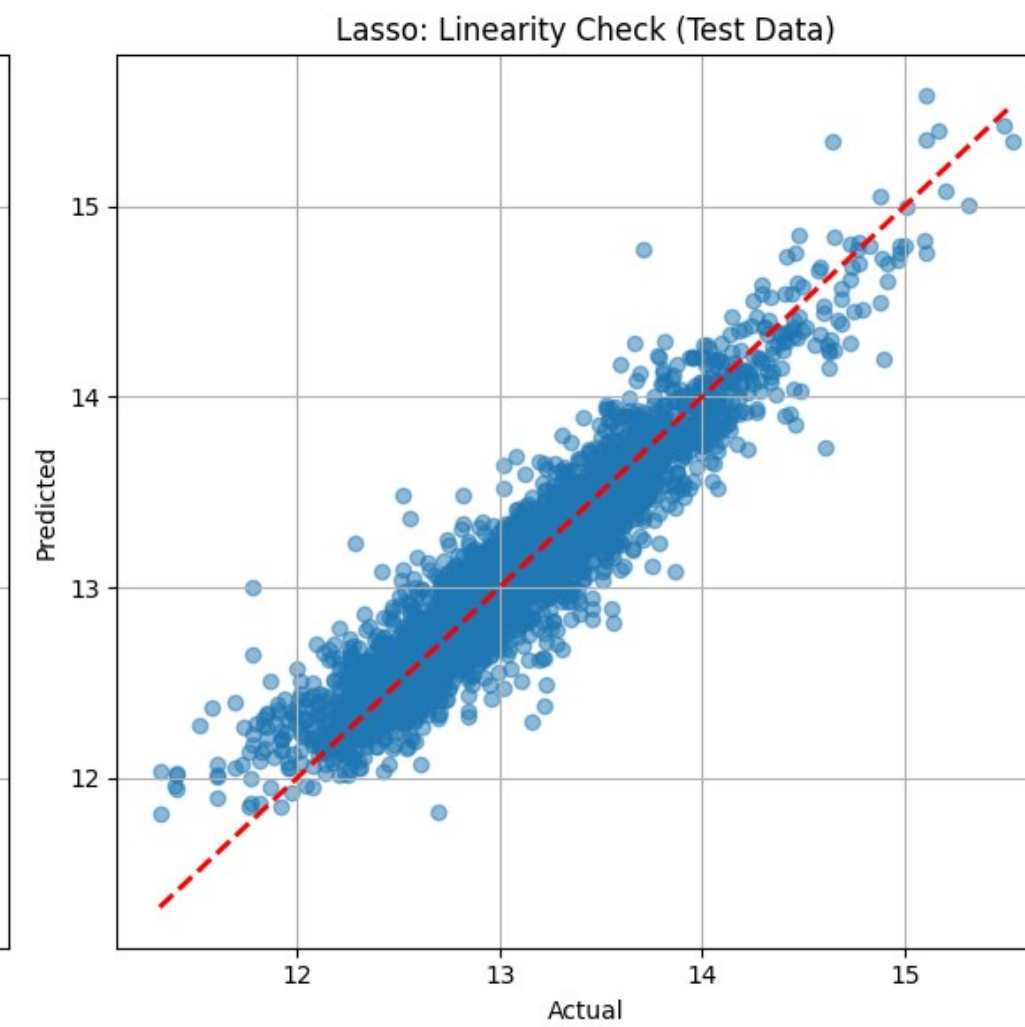
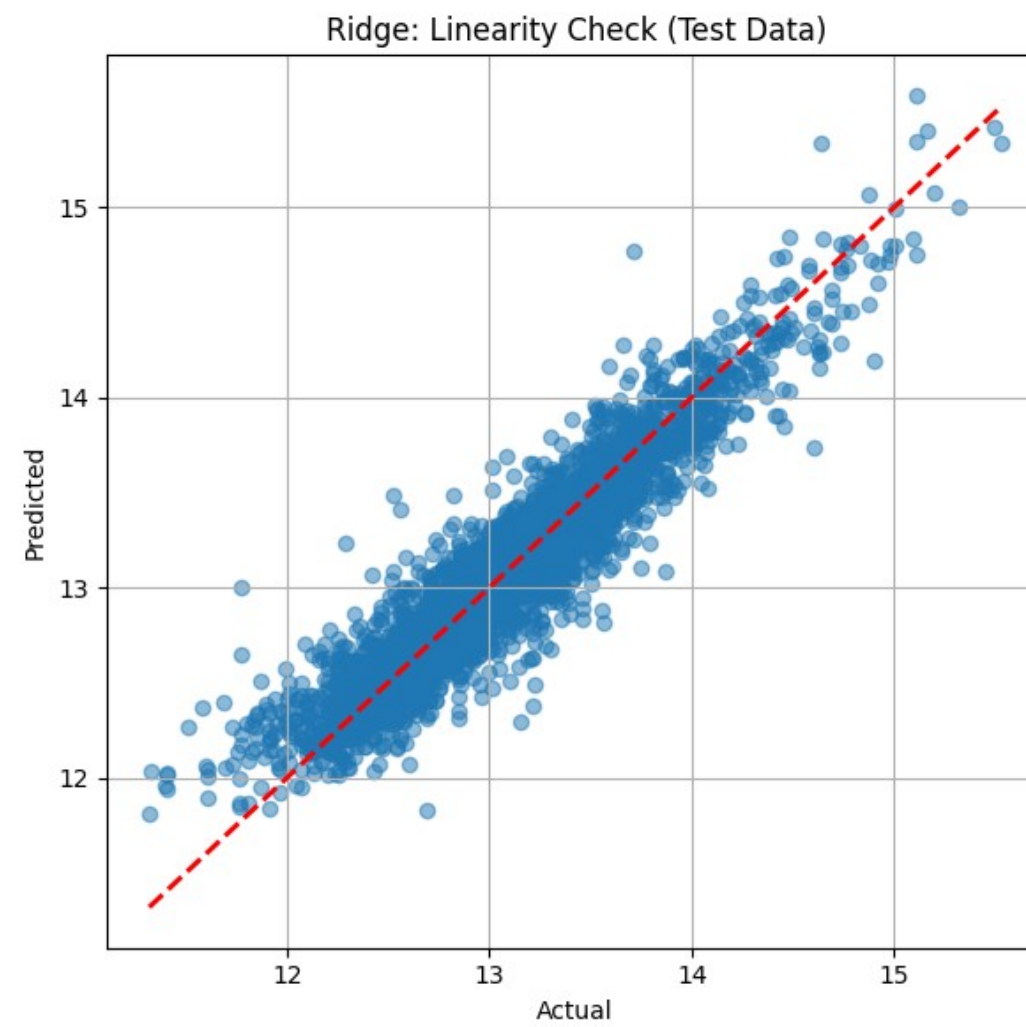
```

5 잔차 선형성 검증

- 테스트 데이터의 실제값과 Ridge 모델의 예측값을 산점도로 찍고 실제값의 최소에서 최대 구간에 걸쳐 $y=x$ 형태의 빨간색 점선을 그림
- Lasso 모델의 예측값도 위와 동일한 방식으로 그림

선형 회귀 분석

Ridge + Lasso



선형 회귀 분석

Ridge + Lasso

```

28 # ----- 2. 잔차의 정규성 (Q-Q Plot) -----
29 plt.figure(figsize=(12, 6))
30 plt.subplot(1, 2, 1)
31 stats.probplot(residuals_train_ridge, dist="norm", plot=plt)
32 plt.title('Ridge: Q-Q Plot of Residuals')
33 plt.grid(True)
34
35 plt.subplot(1, 2, 2)
36 stats.probplot(residuals_train_lasso, dist="norm", plot=plt)
37 plt.title('Lasso: Q-Q Plot of Residuals')
38 plt.grid(True)
39 plt.tight_layout()
40 plt.savefig("ridge_lasso_qq_plot.png")
41 print(" 'ridge_lasso_qq_plot.png' (Q-Q 플롯)이 생성되었습니다.")
42 # plt.show()

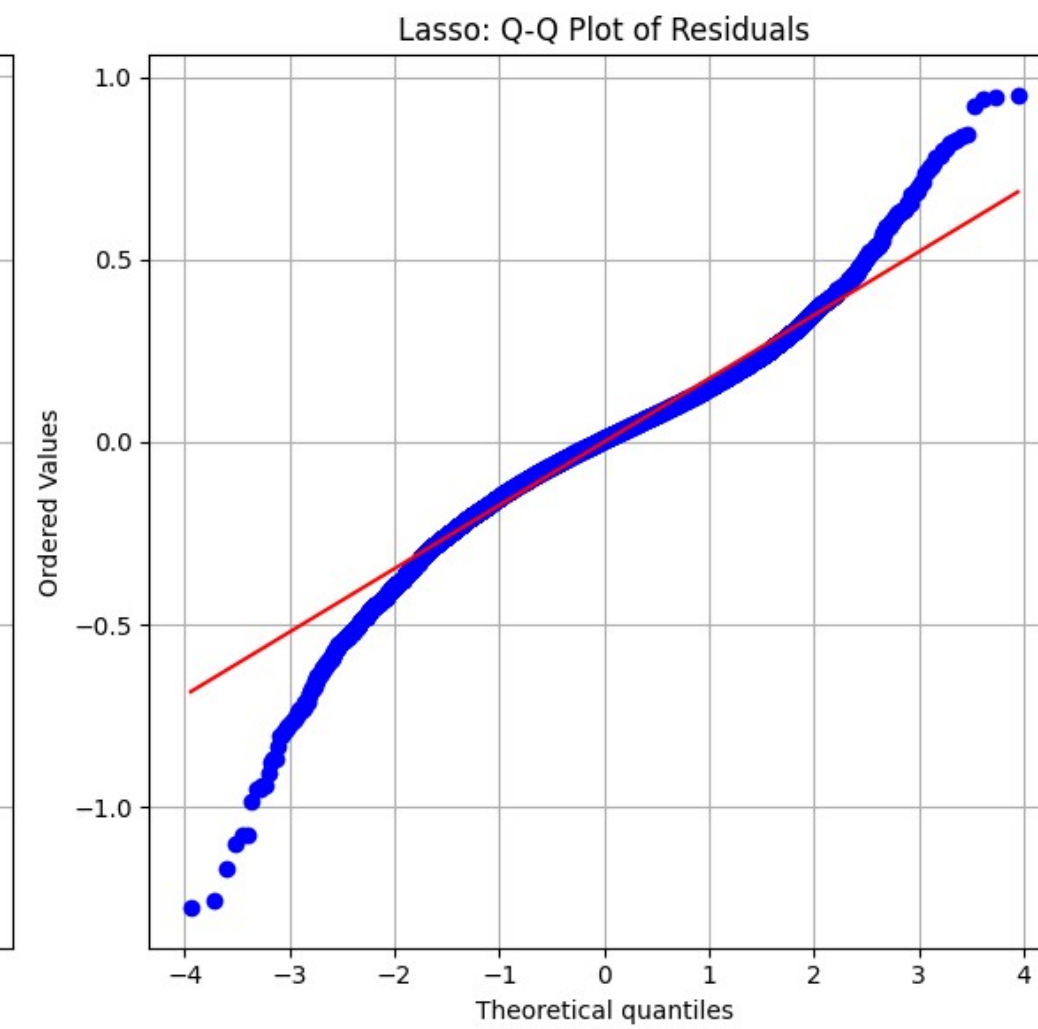
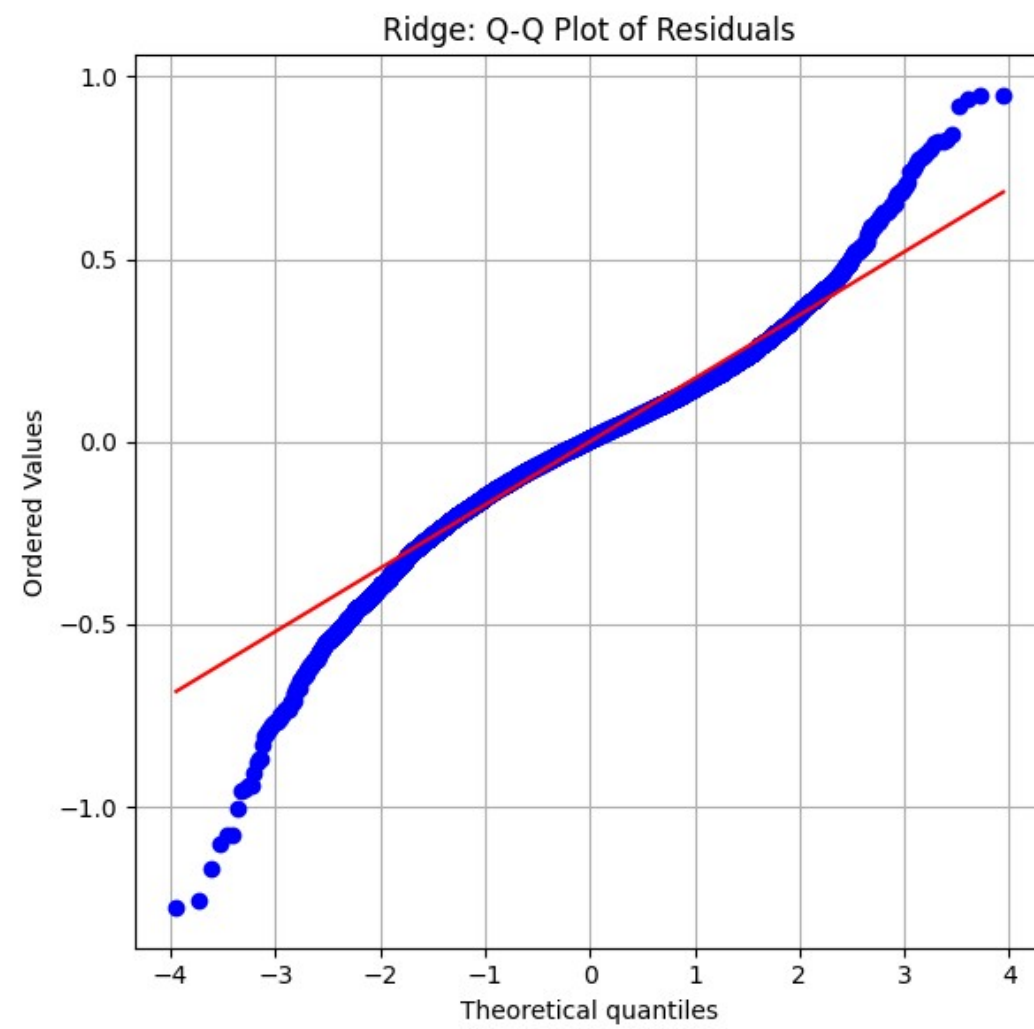
```

6 잔차 정규성 검증

- 왼쪽에 Ridge 모델 잔차를 이용해 Q-Q 플롯을 그림
- 오른쪽에 Lasso 모델 잔차를 이용해 Q-Q 플롯을 그림

선형 회귀 분석

Ridge + Lasso



선형 회귀 분석

Ridge + Lasso

```

45 # ----- 3. 잔차의 등분산성 (Residual Plot) -----
46 plt.figure(figsize=(12, 6))
47 plt.subplot(1, 2, 1)
48 plt.scatter(fitted_values_train_lasso, residuals_train_lasso, alpha=0.5)
49 plt.axhline(y=0, color='r', linestyle='--', lw=2)
50 plt.title('Lasso: Homoscedasticity Check')
51 plt.xlabel('Fitted Values')
52 plt.ylabel('Residuals')
53 plt.grid(True)
54
55 plt.subplot(1, 2, 2)
56 plt.scatter(fitted_values_train_lasso, residuals_train_lasso, alpha=0.5)
57 plt.axhline(y=0, color='r', linestyle='--', lw=2)
58 plt.title('Lasso: Homoscedasticity Check')
59 plt.xlabel('Fitted Values')
60 plt.ylabel('Residuals')
61 plt.grid(True)
62 plt.tight_layout()
63 plt.savefig("ridge_lasso_residual_plot.png")
64 print(" 'ridge_lasso_residual_plot.png' (잔차 플롯)이 생성되었습니다.")
65 # plt.show()

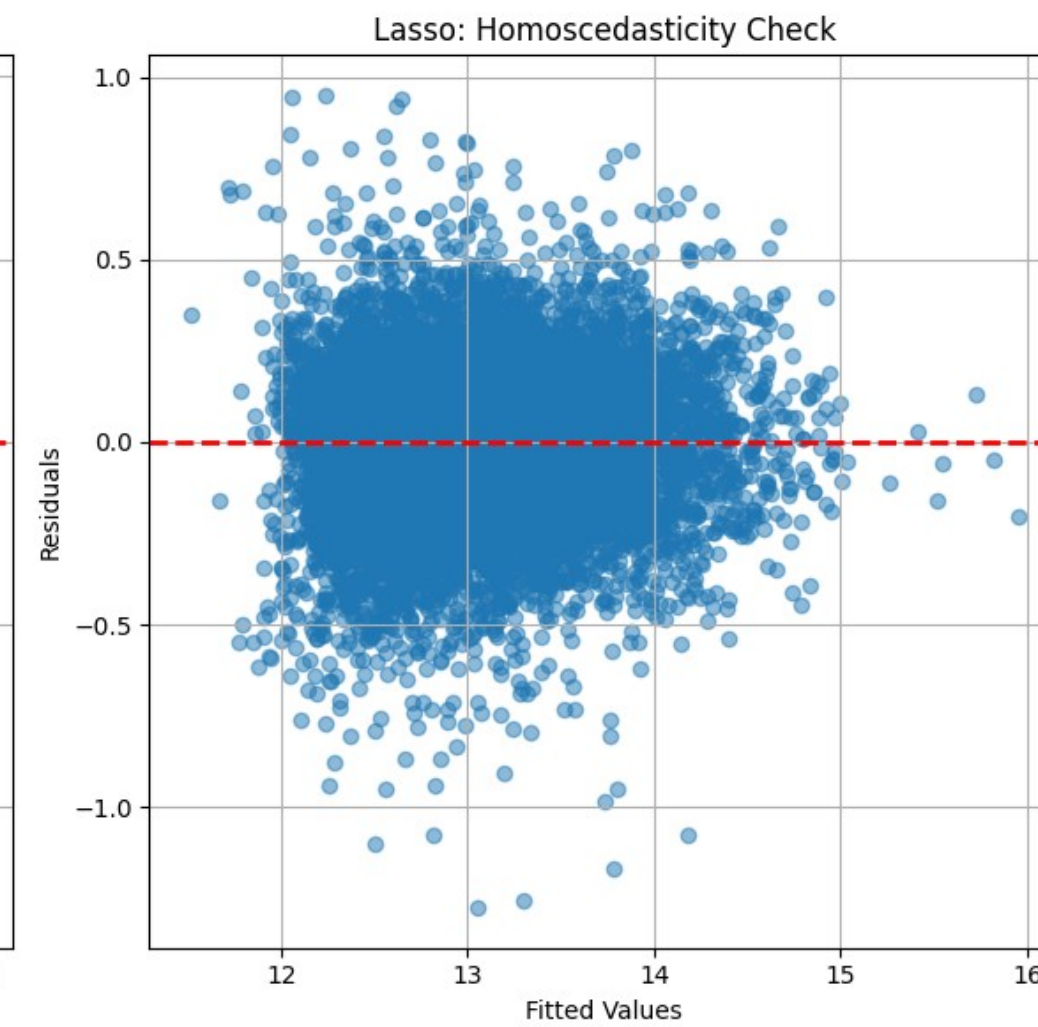
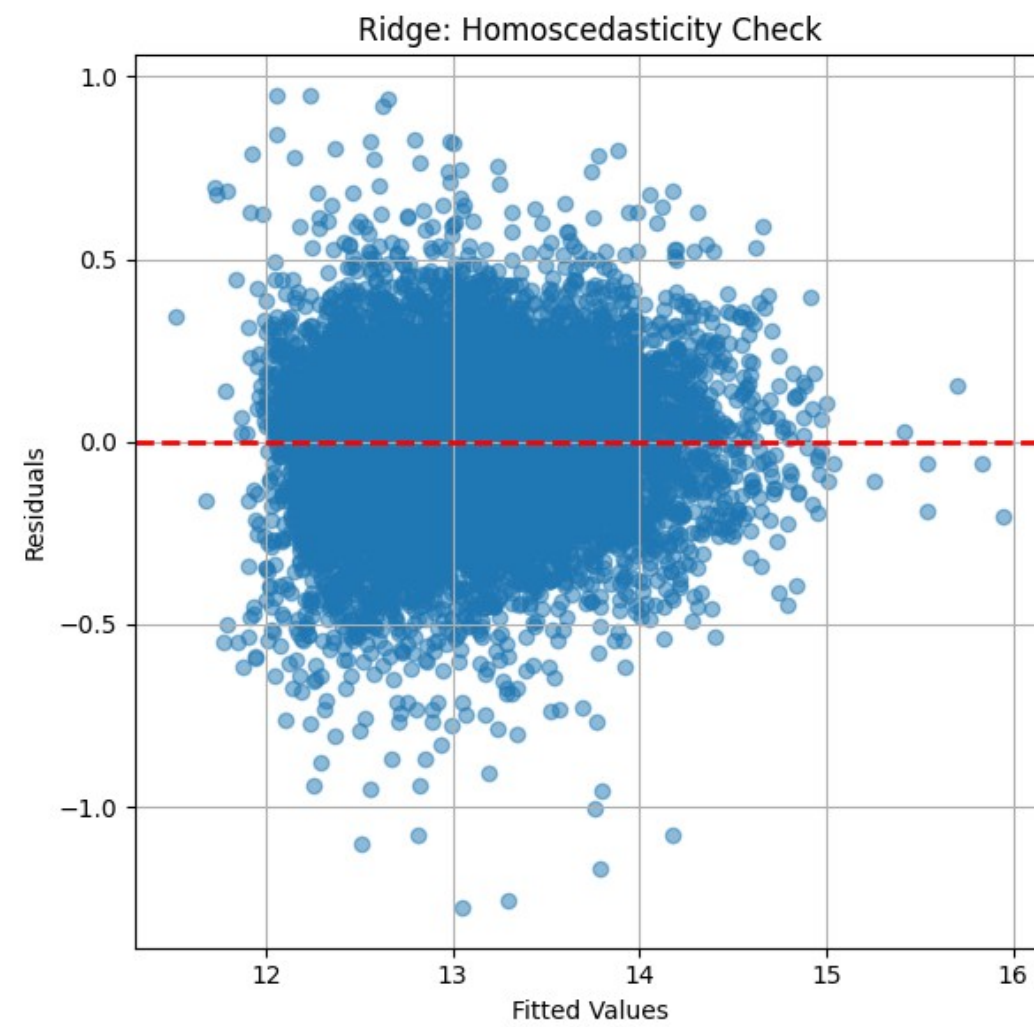
```

7 잔차 등분산성 검증

- 왼쪽 칸의 X축에는 Ridge의 예측값, Y축에는 Ridge의 잔차를 생성하고 잔차의 기준 $y=0$ 에 빨간 점선을 그음
- Lasso 모델의 예측값과 잔차를 동일하게 그림

선형 회귀 분석

Ridge + Lasso



선형 회귀 분석

Ridge + Lasso

```
# ----- 4. 다중공선성 (VIF) -----
print("\n--- 4. 다중공선성(VIF) 검증 ---")
print("VIF는 모델이 아닌 '입력 데이터(X)'의 특성이므로 Ridge/Lasso에 대해 별도로 계산하지 않습니다.")
print("이전 OLS에서 사용한 데이터의 VIF(13.5)가 이 모델들의 진단 결과입니다.")
print("Ridge와 Lasso는 이 높은 VIF를 '제어'하기 위해 사용된 '해결책'입니다.")

print("\n--- 모든 검증 플롯 생성 완료 ---")
```

8 다중공선성 검증

- 다중공선성을 나타내는 VIF 지수는 독립변수 X 데이터 자체가 가진 특성임
- 그러므로 OLS 모델로 풀든, Ridge나 Lasso로 풀든 입력 데이터가 같다면 VIF 값은 동일함

머신러닝

머신러닝 회귀 모델

```

1  # 머신러닝 회귀 모델 4개
2  # LightGBM / CatBoost / XGBoost / RandomForest
3
4  import numpy as np
5  import pandas as pd
6
7  from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
8  from sklearn.ensemble import RandomForestRegressor
9
10 # 결과 저장용
11 model_results = []
12
13 # 평가 함수
14 def evaluate_model(model_name, y_true, y_pred):
15     r2 = r2_score(y_true, y_pred)
16     rmse = np.sqrt(mean_squared_error(y_true, y_pred))
17     mae = mean_absolute_error(y_true, y_pred)
18
19     model_results.append({
20         "Model": model_name,
21         "R2": r2,
22         "RMSE": rmse,
23         "MAE": mae
24     })
25
26     print(f"#n--- {model_name} ---")
27     print(f"R2 Score : {r2:.4f}")
28     print(f"RMSE      : {rmse:.4f}")
29     print(f"MAE       : {mae:.4f}")

```

1 평가 함수 정의

- 'evaluate_model' 이라는 평가 자동화 함수를 만들어서 모델 이름, 실제 정답, 모델의 예측값을 넣으면 앞서 말한 세 가지 지표를 계산하여 저장하게 함

머신러닝

머신러닝 회귀 모델

```

1  # 1. Random Forest Regressor
2
3  rf_model = RandomForestRegressor(
4      n_estimators=300,
5      max_depth=None,
6      min_samples_split=2,
7      min_samples_leaf=1,
8      random_state=42,
9      n_jobs=-1
10 )
11
12 rf_model.fit(X_train, y_train)
13
14 rf_pred = rf_model.predict(X_test)
15
16 evaluate_model("Random Forest", y_test, rf_pred)

```

--- Random Forest ---
 R2 Score : 0.8908
 RMSE : 0.1767
 MAE : 0.1257

2 RandomForest

- 수많은 의사결정 나무를 만들고 그 결과들의 평균을 내어 예측하는 앙상블 모델인 Random Forest를 학습함
- R2 Score가 0.8908로 기존 OLS 점수보다 성능이 살짝 더 좋아짐

머신러닝

머신러닝 회귀 모델

```

1  # 2. XGBoost Regressor
2
3  from xgboost import XGBRegressor
4
5  xgb_model = XGBRegressor(
6      n_estimators=500,
7      learning_rate=0.05,
8      max_depth=5,
9      subsample=0.8,
10     colsample_bytree=0.8,
11     objective='reg:squarederror',
12     random_state=42,
13     n_jobs=-1
14 )
15
16 xgb_model.fit(X_train, y_train)
17
18 xgb_pred = xgb_model.predict(X_test)
19
20 evaluate_model("XGBoost", y_test, xgb_pred)

```

```

--- XGBoost ---
R2 Score : 0.9113
RMSE     : 0.1593
MAE      : 0.1135

```

3 XGBoost

- 앞선 나무가 틀린 문제를 다음 나무가 집중적으로 고쳐나가는 방식인 XGBoost를 학습시킴
- 학습 및 예측 후 채점 결과 R2 Score가 0.9113으로 Random Forest보다 뛰어난 성능을 보여줌

머신러닝

머신러닝 회귀 모델

```

1  # 3. LightGBM Regressor
2
3  from lightgbm import LGBMRegressor
4
5  lgbm_model = LGBMRegressor(
6      n_estimators=500,
7      learning_rate=0.05,
8      max_depth=-1,
9      num_leaves=31,
10     subsample=0.8,
11     colsample_bytree=0.8,
12     random_state=42,
13     n_jobs=-1
14 )
15
16 lgbm_model.fit(X_train, y_train)
17
18 lgbm_pred = lgbm_model.predict(X_test)
19
20 evaluate_model("LightGBM", y_test, lgbm_pred)

```

```

--- LightGBM ---
R2 Score : 0.9141
RMSE      : 0.1567
MAE       : 0.1117

```

4 LightGBM

- XGBoost보다 학습 속도가 훨씬 빠르면서 성능도 비슷한 LightGBM을 학습시킴
- 학습 및 예측 결과 R2 Score가 0.9141로 현재까지 중 가장 높은 예측력을 기록함

머신러닝

머신러닝 회귀 모델

```

1  # 4. CatBoost Regressor
2  !pip install catboost
3  from catboost import CatBoostRegressor
4
5  cat_model = CatBoostRegressor(
6      iterations=500,
7      learning_rate=0.05,
8      depth=6,
9      loss_function='RMSE',
10     random_seed=42,
11     verbose=0
12 )
13
14 cat_model.fit(X_train, y_train)
15
16 cat_pred = cat_model.predict(X_test)
17
18 evaluate_model("CatBoost", y_test, cat_pred)

```

```

--- CatBoost ---
R2 Score : 0.9111
RMSE      : 0.1594
MAE       : 0.1145

```

5 CatBoost

- 범주형 데이터 처리에 매우 강력하고 과적합 방지에 탁월한 CatBoost를 학습시킴
- 학습 및 예측 결과 R2 Score가 0.9111로 XGBoost와 거의 유사하게 우수한 성능을 보여줌

머신러닝

머신러닝 회귀 모델

```

1  #성능 비교
2
3  results_df = pd.DataFrame(model_results)
4
5  results_df = results_df.sort_values(by="R2", ascending=False)
6
7  print("\n" + "="*60)
8  print("모델 성능 비교")
9  print("="*60)
10 print(results_df)

```

```

=====
모델 성능 비교
=====

```

	Model	R2	RMSE	MAE
2	LightGBM	0.914062	0.156744	0.111715
1	XGBoost	0.911277	0.159263	0.113501
3	CatBoost	0.911096	0.159426	0.114521
0	Random Forest	0.890827	0.176667	0.125693

6 모델의 성능 비교

- 지금까지 학습한 4개 머신러닝 모델의 채점 결과표를 정렬하여 출력함
- LightGBM이 R2 Score 0.914062, RMSE 0.1567로 오차가 가장 적고 가격을 가장 정확하게 예측함

머신러닝

머신러닝 회귀 모델

```

1  import matplotlib.pyplot as plt
2
3  # 변수 중요도 확인
4
5  def plot_feature_importance(model, model_name, X_train, top_n=20):
6      importance = model.feature_importances_
7      feature_names = X_train.columns
8
9      fi_df = pd.DataFrame({
10         "Feature": feature_names,
11         "Importance": importance
12     }).sort_values(by="Importance", ascending=False)
13
14     print(f"\n--- {model_name} 변수 중요도 Top {top_n} ---")
15     print(fi_df.head(top_n))
16
17     plt.figure(figsize=(10, 6))
18     plt.barh(fi_df.head(top_n)["Feature"][:top_n], fi_df.head(top_n)["Importance"][:top_n])
19     plt.title(f"{model_name} Feature Importance")
20     plt.xlabel("Importance")
21     plt.ylabel("Feature")
22     plt.tight_layout()
23     plt.show()
24
25     plot_feature_importance(rf_model, "Random Forest", X_train)
26     plot_feature_importance(xgb_model, "XGBoost", X_train)
27     plot_feature_importance(lgbm_model, "LightGBM", X_train)
28     plot_feature_importance(cat_model, "CatBoost", X_train)

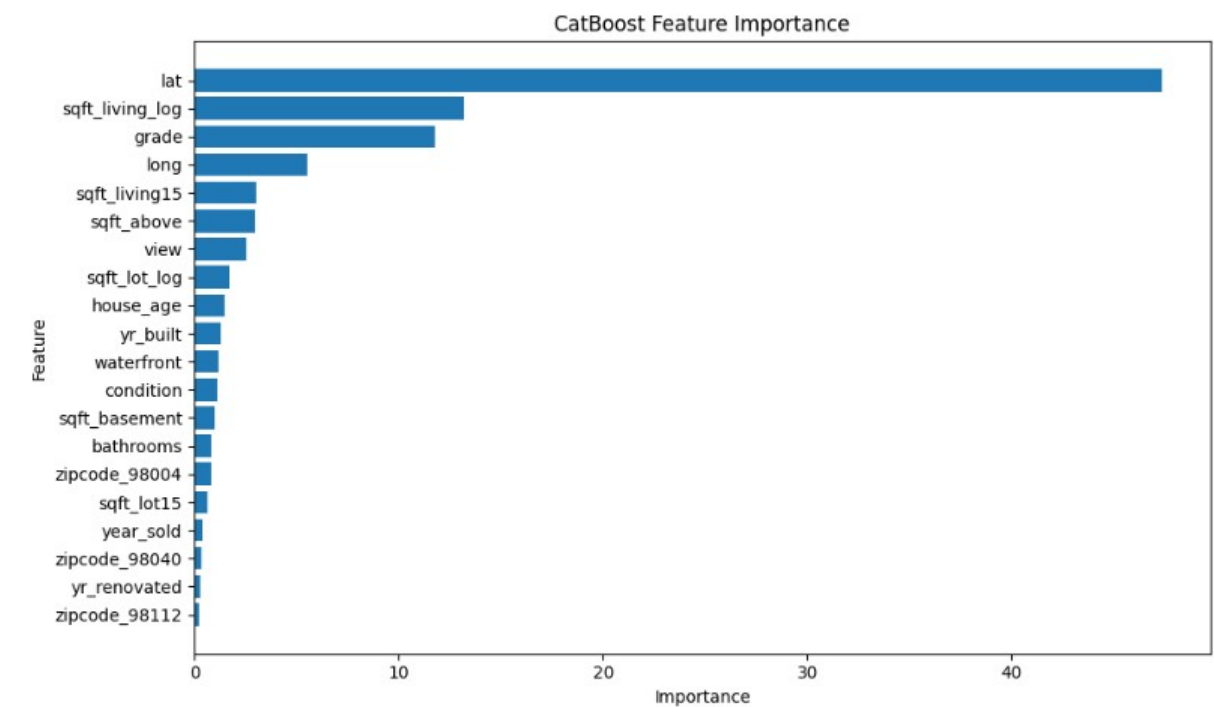
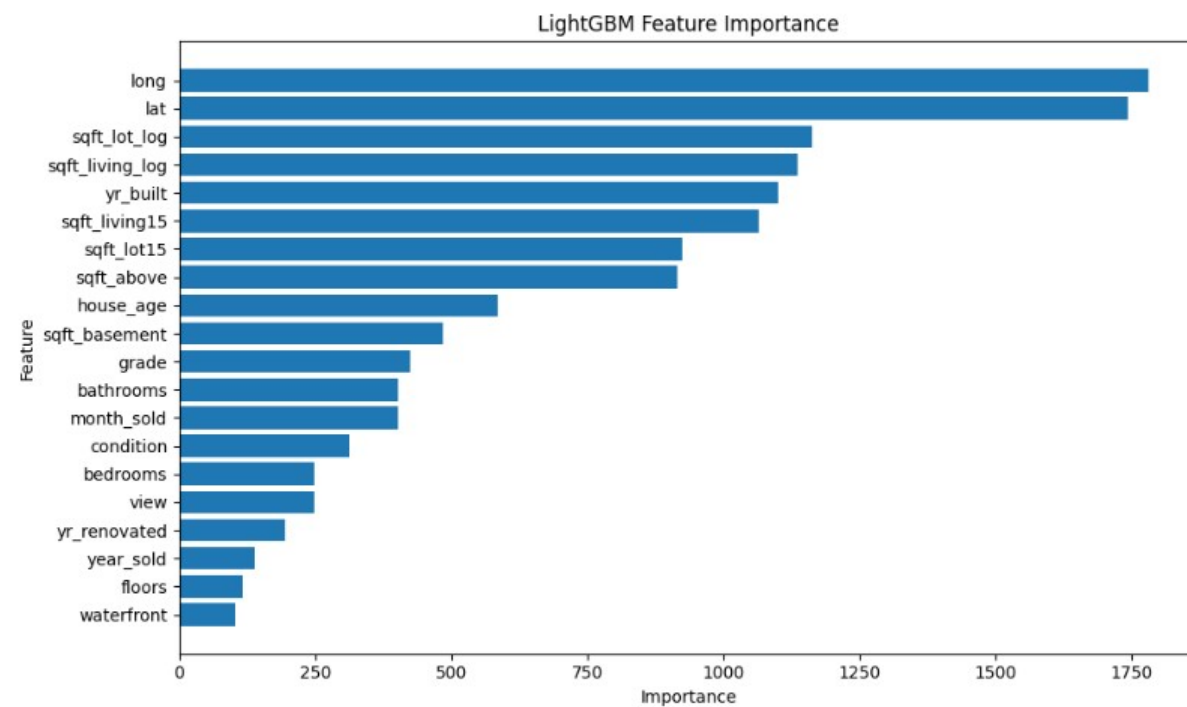
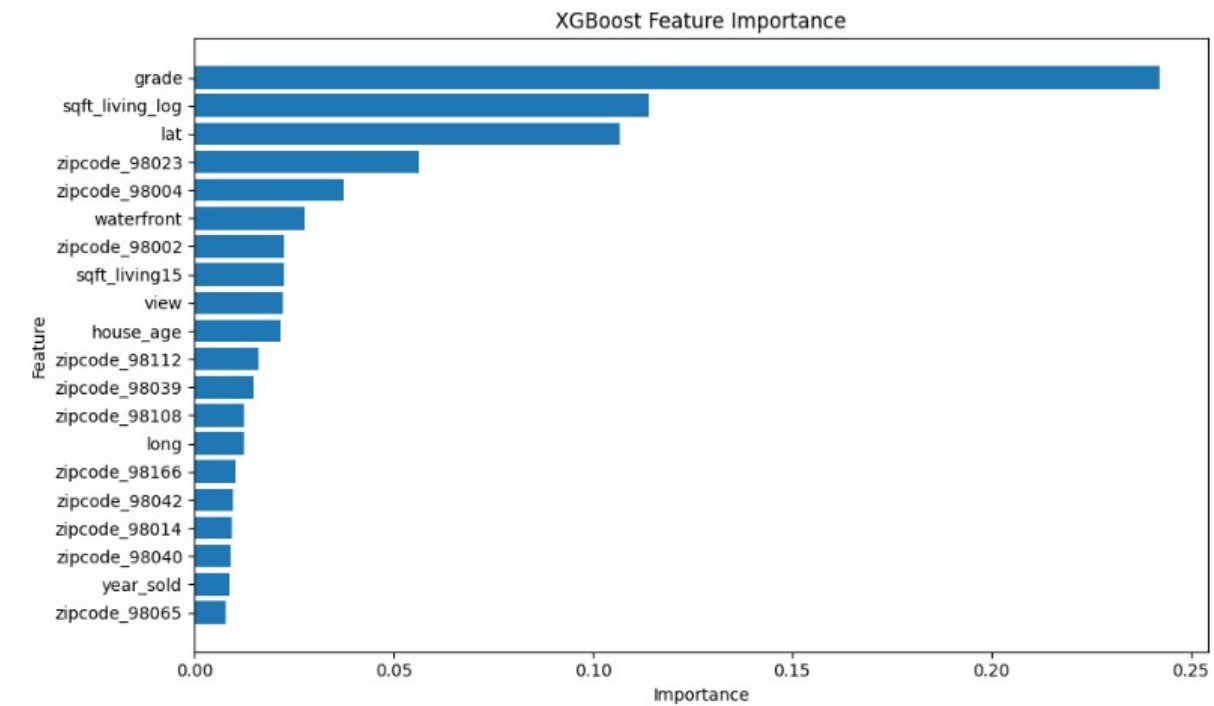
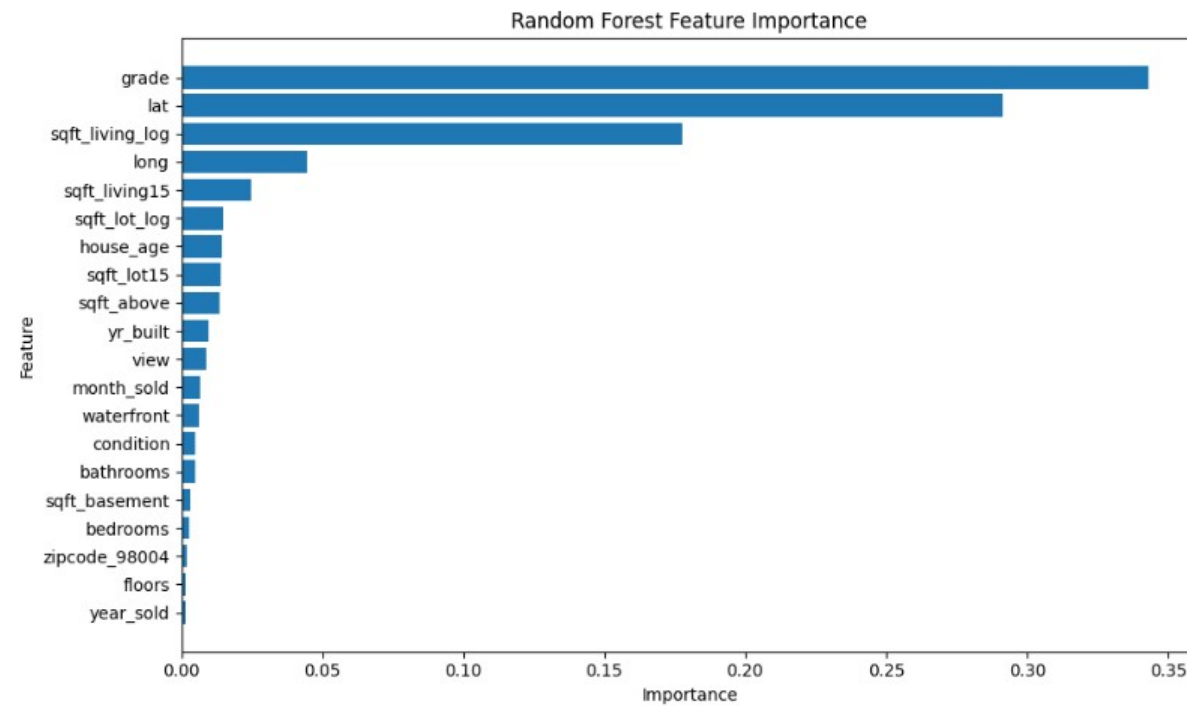
```

7 중요도 시각화

- 각 모델마다 중요도를 그리는 코드를 반복하지 않도록 plot_feature_importance 라는 자동화 함수를 만들고 모델, 훈련 데이터 등을 입력받음

머신러닝

머신러닝 회귀 모델



분석 마무리

최적의 예측 모델 도출 및 성능 검증

1 OLS의 한계

초기에 구축했던 다중 선형 회귀(OLS) 모델은 설명력 88.8%로 유의미한 기준점을 제시했으나, 주택 시장 데이터에 내재된 복잡한 비선형적 패턴을 모두 반영하기에는 한계가 존재함

2 앙상블 모델의 도입

이를 극복하기 위해 다중공선성 문제를 완화하는 규제 모델 Ridge, Lasso와 트리 기반 앙상블 모델 4종 (Random Forest, XGBoost, LightGBM, CatBoost) 을 도입하여 교차 검증을 수행함

3 최적 모델 도출

결론적으로, 앞 중심의 분할 방식을 통해 세부적인 데이터 패턴을 가장 빠르게 포착한 LightGBM 모델이 설명력 91.4%, RMSE 0.156을 기록하며 가장 우수한 예측 성능을 확보했음

분석 마무리

주택 가격 결정의 핵심 변수 도출

1 가격 결정 핵심 요인

최우수 앙상블 모델들의 변수 중요도를 종합적으로 분석한 결과, 주택 가격을 결정짓는 핵심 요인은 크게 물리적 스펙, 세부 입지 조건 두 축으로 나뉘는 것으로 확인됨

2 물리적 스펙

시(City)에서 평가한 전반적인 건축 및 디자인 등급(grade)과 로그 변환된 실주거 면적(sqft_living_log)이 가격의 기본적인 기준선을 형성하는 가장 큰 요인으로 작용함

3 세부 입지 조건

위도(lat)와 경도(long), 그리고 특정 우편번호와 수변 조망권(waterfront) 변수가 상위권을 차지함.
이는 단순히 '어느 도시에 있는가'를 넘어 특정 블록 단위의 지리적 위치가 주택의 가치를 결정짓는 핵심임을 의미함

결론

결론 및 비즈니스 시사점

1 부동산 가치의 핵심

분석 결과 '입지가 가장 중요하다'는 부동산 시장의 전통적인 경험칙이 머신러닝의 변수 중요도를 통해 통계적으로 입증됨. 또한 OLS 모델의 계수 분석에서 확인되었듯, 제한된 면적 내에서 방이나 층수를 늘리는 단순한 공간 분할은 오히려 주택 가치에 부정적인 영향을 미쳤습니다. 이는 향후 부동산 개발 및 리모델링 전략 수립 시, 높은 건축 퀄리티 유지와 쾌적한 공간감 확보가 부동산 가치 상승에 유리함을 시사함.

2 비즈니스 활용 방안

도출된 LightGBM 기반의 예측 모델은 부동산 중개 플랫폼에서 고객에게 적정 매물가를 제시하는 자동 가치 산정 모델의 핵심 알고리즘으로 활용될 수 있음. 특정 우편번호와 위도 및 경도에 따른 가격 민감도를 활용하여, 부동산 투자 및 매입 시 저평가된 우량 지역을 발굴하는 정량적 지표로 사용 가능함.

3 연구의 한계점

본 연구의 데이터는 미국 King County 라는 특정 지역의 과거 거래 내역에 국한되어 있어 타 지역의 부동산 시장에 모델을 그대로 적용하기에는 무리가 있음. 또한 주택 가격 변동에 결정적인 영향을 미치는 기준 금리, 대출 규제, 인플레이션 등 거시경제 데이터가 반영되지 않았음. 향후 이 모델에 시계열 경제 지표를 추가 결합한다면 시장 상황 변화에 선제적으로 대응할 수 있는 더욱 고도화된 예측 시스템으로 발전할 수 있을 것이라 기대함.